

Software Engineering

Complex Engineering Problem Report: Prison Management System Design

Submitted by

Shawaiz Asghar

BSCE22036

Muhammad Saad

BSCE22021

Muhammad Miqdad

BSCE22001

Part (a) – Structural and Dynamic Models

1. Structural Model

The structural model defines the static architecture of the Prison Management System, including **class diagrams**, **interfaces**, and **design patterns** that align with the system's use cases.

Class Diagram

The system consists of the following key classes :

Class Name	Responsibility / Description
SecuritySystem	Central security controller managing lockdowns and system activation/deactivation.
AlarmSystem	Triggers alarms, initiates lockdowns, and notifies prison personnel during emergencies.
SurveillanceSystem	Continuously monitors prison areas, detects anomalies, and records surveillance footage.
SecurityOfficer	Monitors security systems, responds to alerts, and performs safety checks.
Warden	Oversees prison protocols, approves administrative decisions, and manages prison staff.
Inmate	Represents prisoners, tracks behavior, assigns work, and interacts with health/rehab systems.
HealthRecord	Maintains medical records, schedules checkups, and handles medical emergencies.
Visitor	Requests appointments to visit inmates and tracks the status of visit requests.
visitSchedule	Manages booking of inmate visitations, approval of visit requests, and pass generation.
User	Authenticated entity that can log in and perform actions based on assigned roles.
Administration	Manages users, configures system settings, and views audit logs.
backgroundcheck	Runs screenings and stores results for visitor or staff verification.
contrabandScanner	Scans individuals/packages for illegal items and flags concerns.
incidentReport	Documents security incidents, assigns severity levels, and notifies stakeholders.
ComplianceReport	Validates regulations, submits records to authorities, and archives them for review.

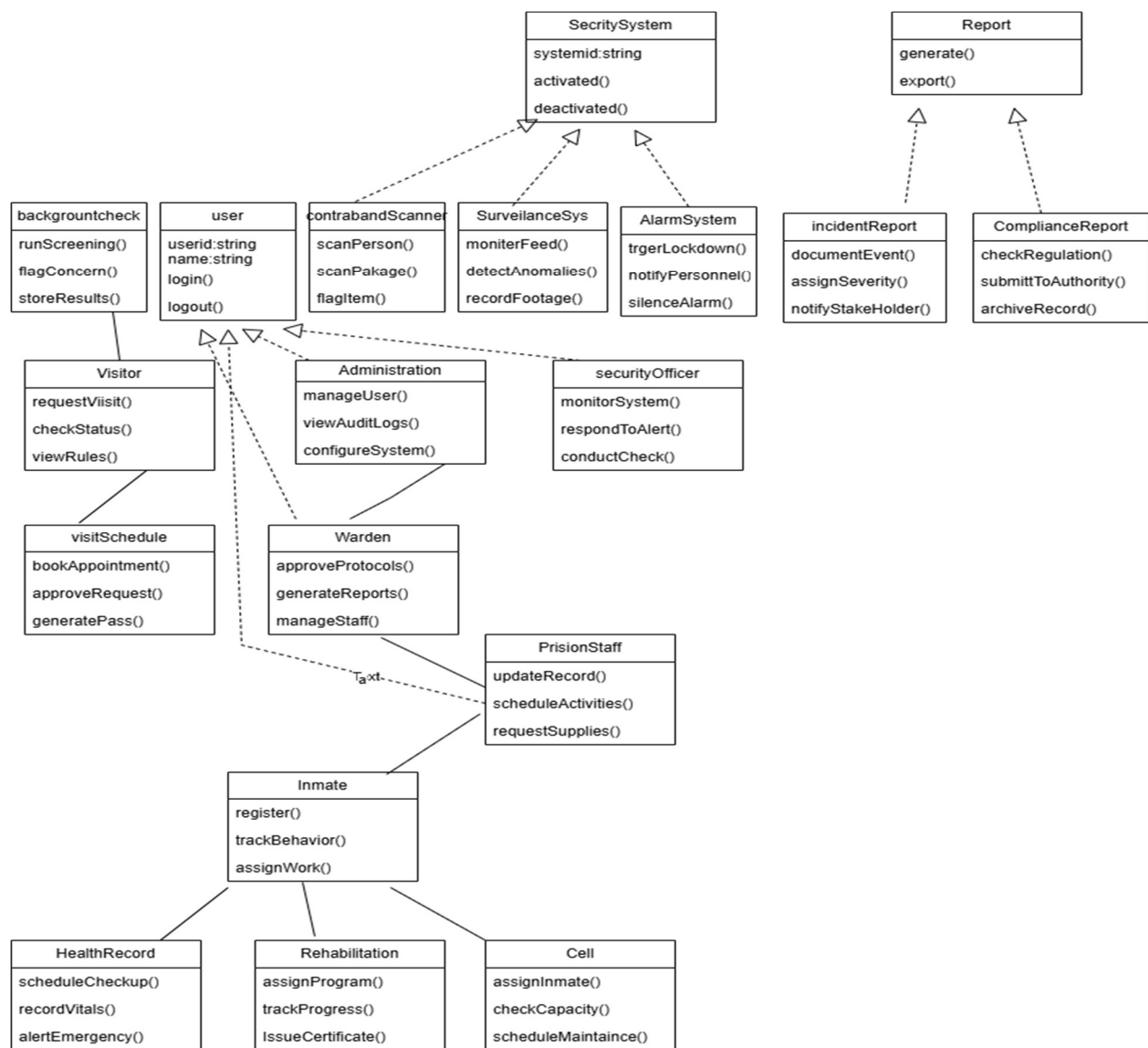
Report Abstract class for generating and exporting different types of reports.

Rehabilitation Assigns inmates to programs, tracks their progress, and issues completion certificates.

PrisonStaff Performs daily operations, updates records, schedules activities, and requests supplies.

Cell Assigns inmates to cells, checks capacity, and schedules maintenance.

MonitoringContext Dynamically sets and executes monitoring strategies via the Monitorable interface.



Interface Summary and Responsibilities

1) SecurityOperation

- Provides essential security functionalities.
- Ensures systems can initiate lockdowns and verify personnel identities.
- Implemented by: AlarmSystem, SurveillanceSystem
- Key methods:
 - lockdown()
 - authenticatePersonnel()

2) Monitorable

- Enables activity monitoring and reporting in the prison system.
- Allows alerting the security team based on priority.
- Implemented by: SurveillanceSystem, (can be used in a Strategy pattern with MonitoringContext)
- Key methods:
 - trackActivity(): LogEntry
 - generateReport(): Report
 - alertSecurity(priority: int): void

3) Authenticable

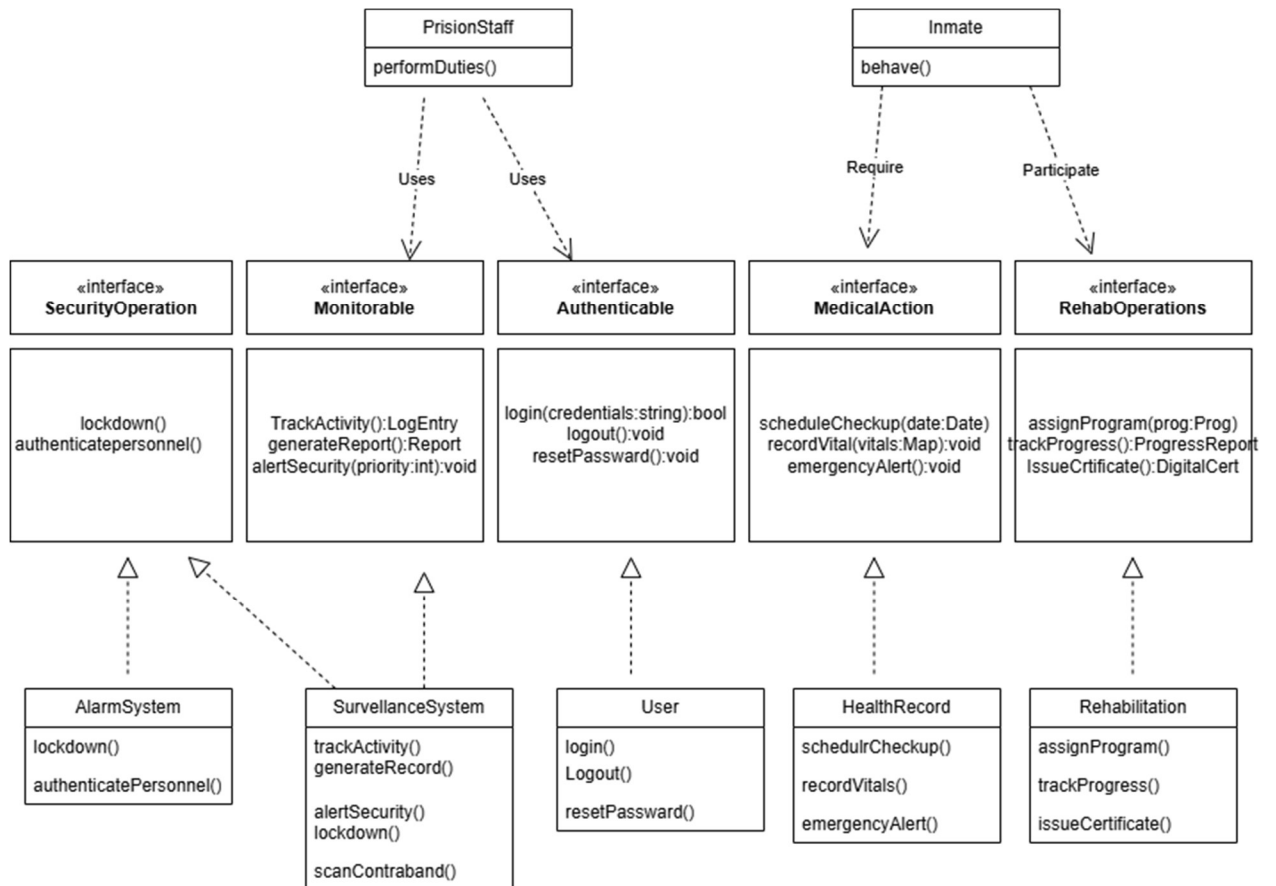
- Supports user authentication and credential management.
- Ensures only authorized access to systems.
- Implemented by: User
- Key methods:
 - login(credentials: string): bool
 - logout(): void
 - resetPassword(): void

4) MedicalAction

- Handles inmate healthcare procedures.
- Manages medical appointments, vitals recording, and emergencies.
- Implemented by: HealthRecord
- Key methods:
 - scheduleCheckup(date: Date): void
 - recordVital(vitals: Map): void
 - emergencyAlert(): void

5) RehabOperations

- Manages inmate rehabilitation programs.
- Tracks progress and issues completion certificates.
- Implemented by: Rehabilitation
- Key methods:
 - assignProgram(prog: Prog)
 - trackProgress(): ProgressReport
 - issueCertificate(): DigitalCert



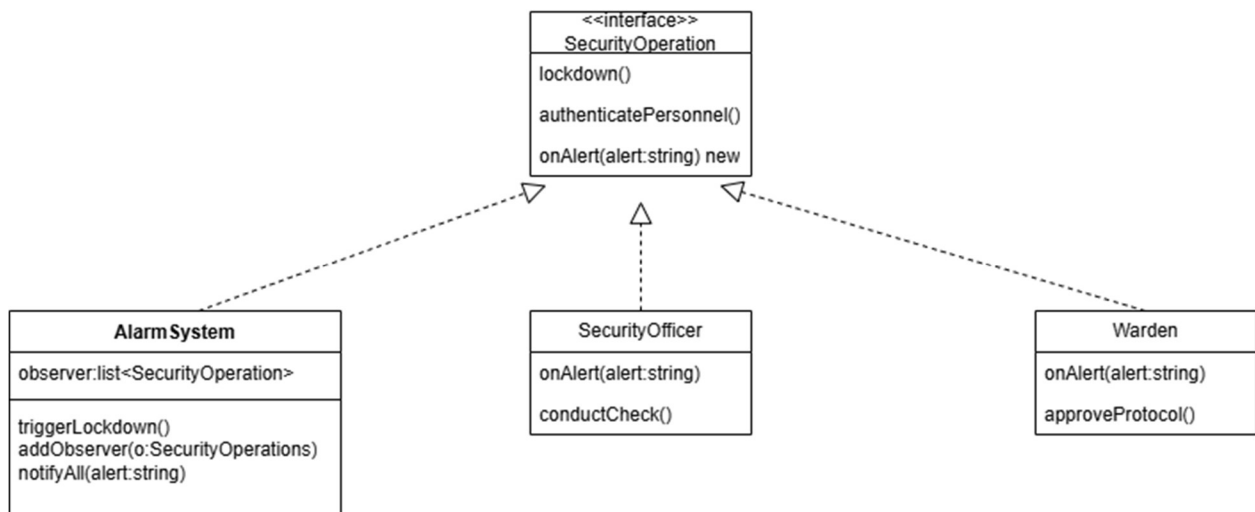
Design Patterns Implementation

The **design patterns** implemented in the Prison Management System, strictly based on the provided **class and interface diagrams**. Each pattern is justified with respect to system requirements and interface usage.

1. Observer Pattern

Implemented in: AlarmSystem, SecurityOfficer, Warden

Interfaces Used: SecurityOperation (modified with onAlert())



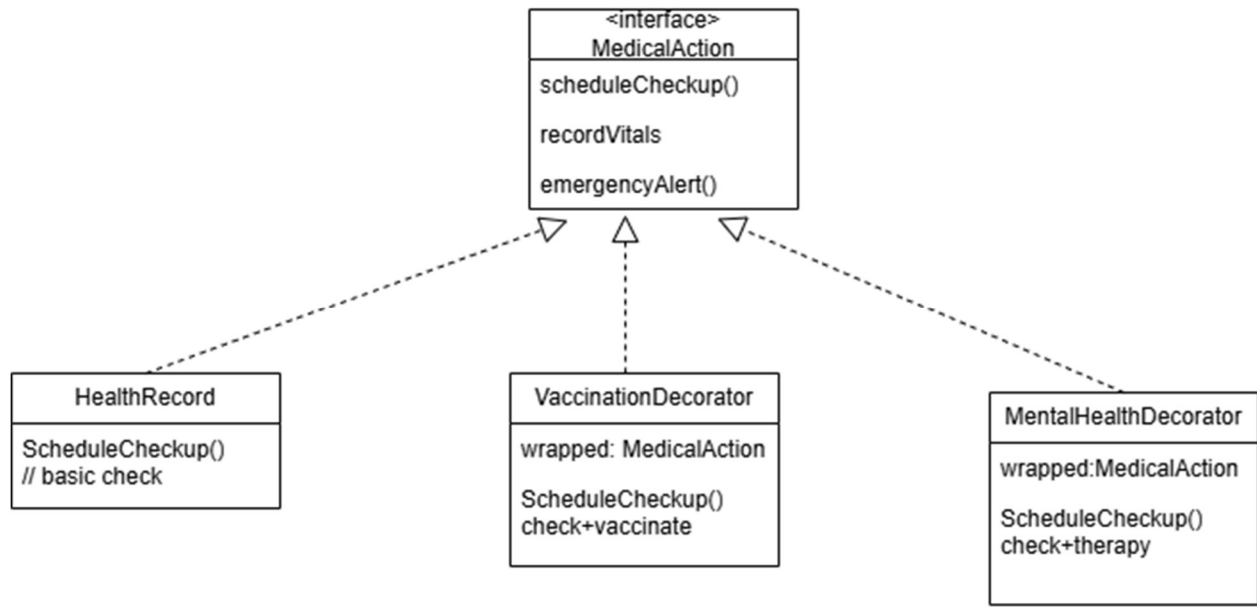
Why Observer?

- **Real-time alerts:** AlarmSystem needs to notify multiple staff types (SecurityOfficer, Warden) without tight coupling.
- **Interface alignment:** Extended SecurityOperation with onAlert() to standardize observer behavior.
- **Scalability:** New observers (e.g., MedicalStaff) can be added without modifying AlarmSystem.

2. Decorator Pattern

Implemented in: HealthRecord, VaccinationDecorator, MentalHealthDecorator

Interfaces Used: MedicalAction

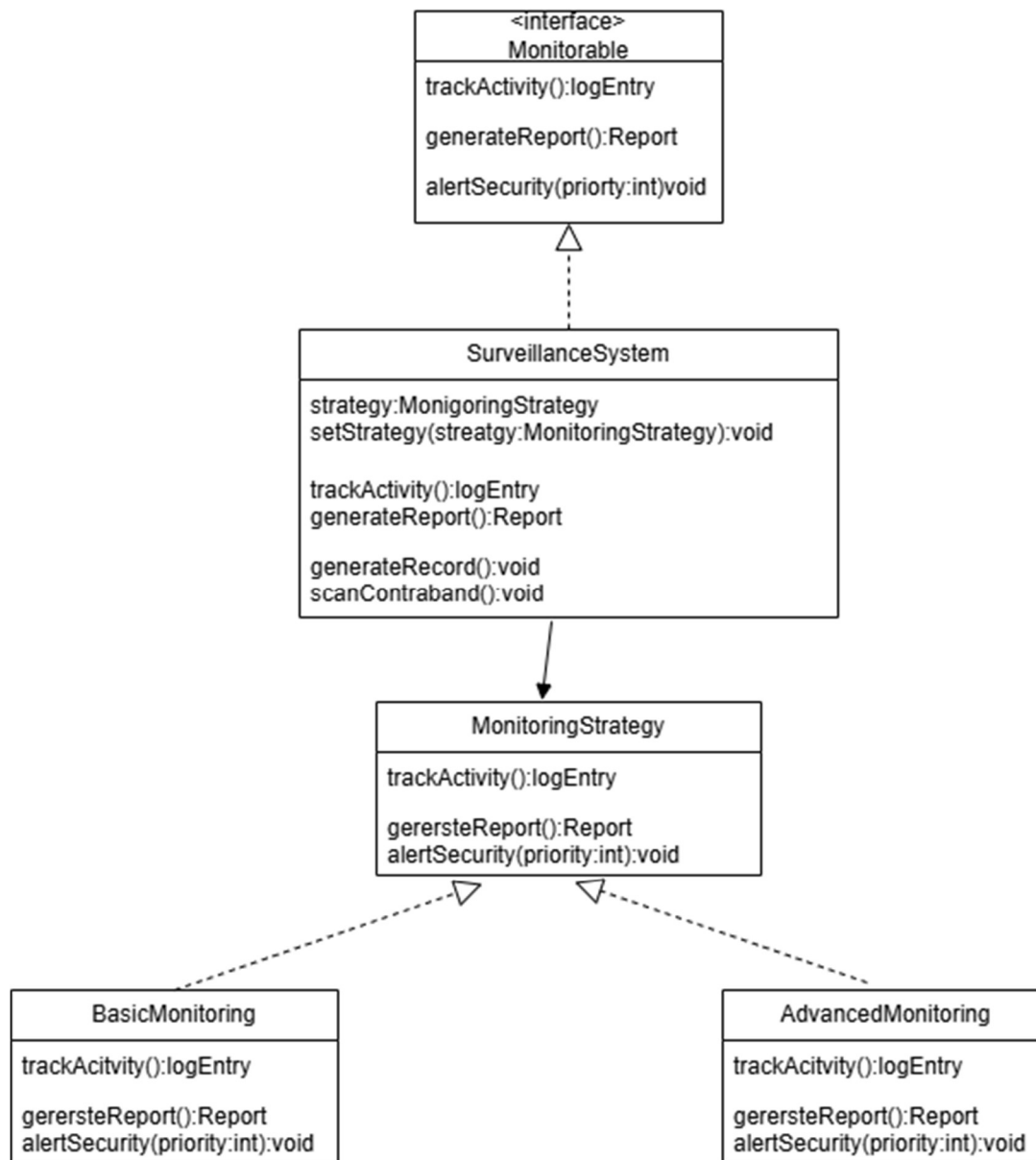


Why Decorator?

- **Extend health services:** Add vaccines/mental health checks to basic checkups **without subclass explosion**.
- **Interface compliance:** All decorators implement **MedicalAction**, ensuring method consistency (`scheduleCheckup()`).
- **Dynamic enhancements:** Stack decorators (e.g., Vaccination + MentalHealth) at runtime.

3. Strategy Pattern

Implemented in: SurveillanceSystem, BasicMonitoring, AdvancedMonitoring
Interfaces Used: Monitorable, MonitoringStrategy



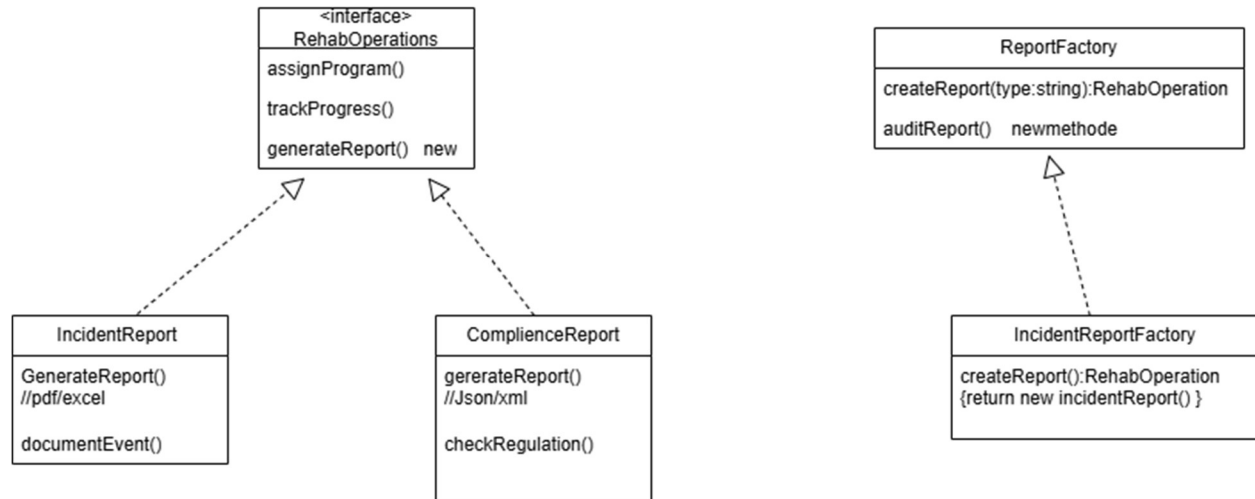
Why Strategy?

- **Interchangeable algorithms:** Switch between BasicMonitoring (logs) and AdvancedMonitoring (AI) dynamically.
- **Interface-driven:** MonitoringStrategy mirrors Monitorable for seamless integration.
- **OCP compliance:** New strategies (e.g., RealTimeMonitoring) can be added without modifying SurveillanceSystem.

4. Factory Method Pattern

Implemented in: ReportFactory, IncidentReport, ComplianceReport

Interfaces Used: RehabOperations (extended with generateReport())



Why Factory Method?

- **Centralized report creation:** Hide implementation details of IncidentReport/ComplianceReport.
- **Interface alignment:** All reports implement `RehabOperations.generateReport()`.
- **Audit trail:** `ReportFactory.auditReport()` ensures compliance without polluting report classes.

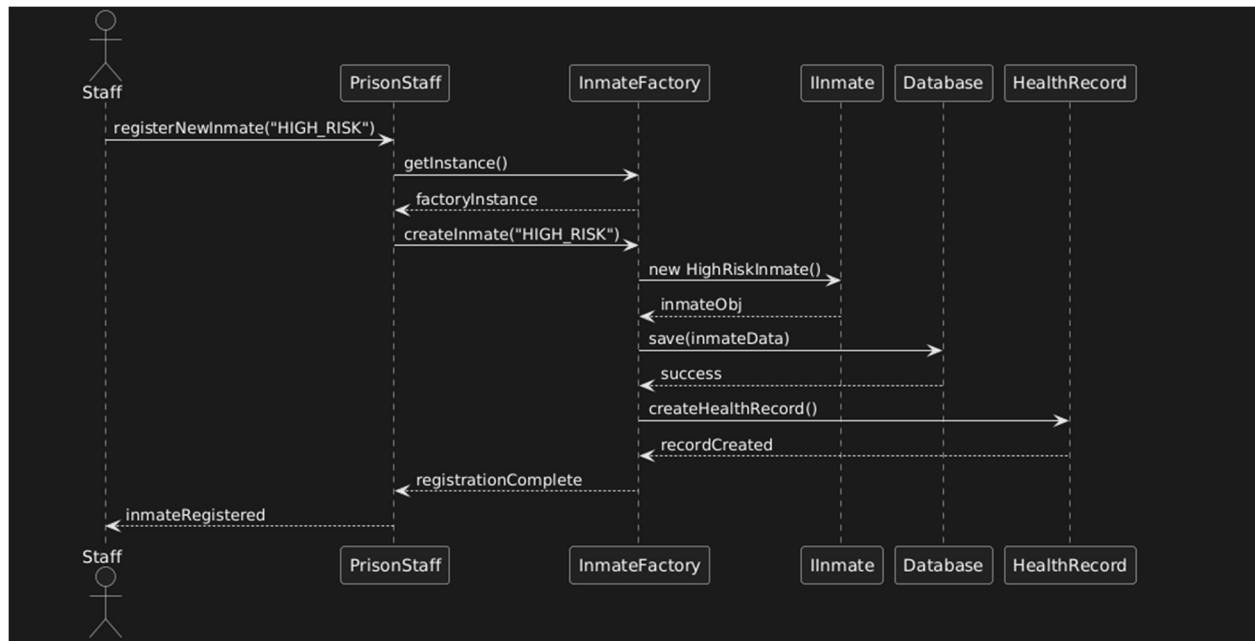
Summary

Pattern	Problem Solved	Interface Usage
Observer	Decoupled alert notifications	Extended SecurityOperation
Decorator	Dynamic health service extensions	MedicalAction
Strategy	Interchangeable monitoring logic	Monitorable → MonitoringStrategy
Factory	Flexible report creation	Extended RehabOperations

Part B: Dynamic Model - Sequence Diagrams

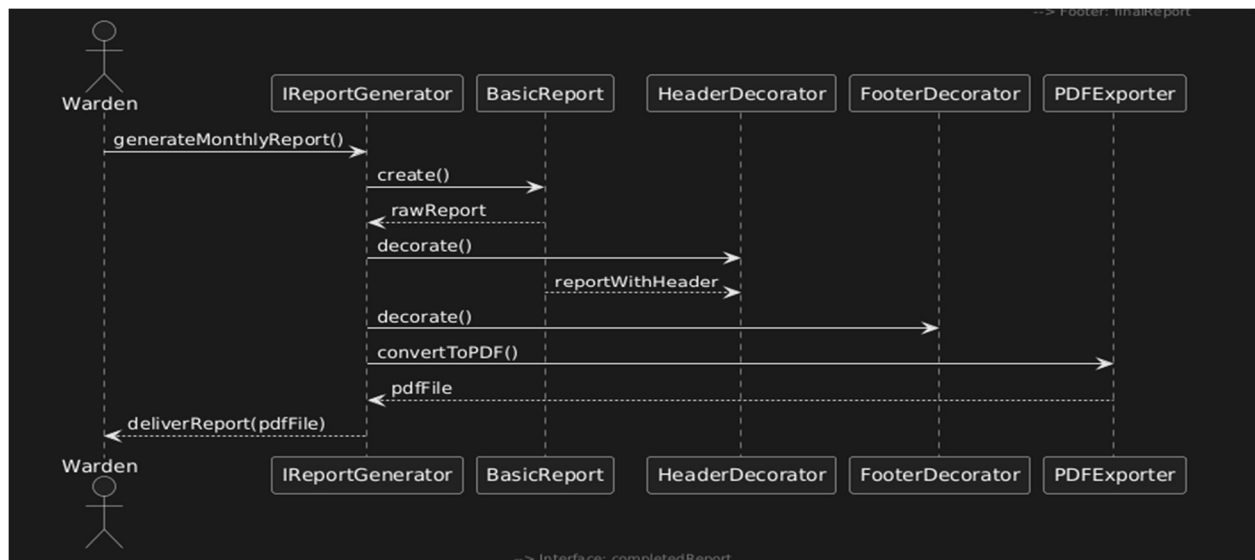
1. Inmate Registration

Actors: PrisonStaff, InmateFactory, Database



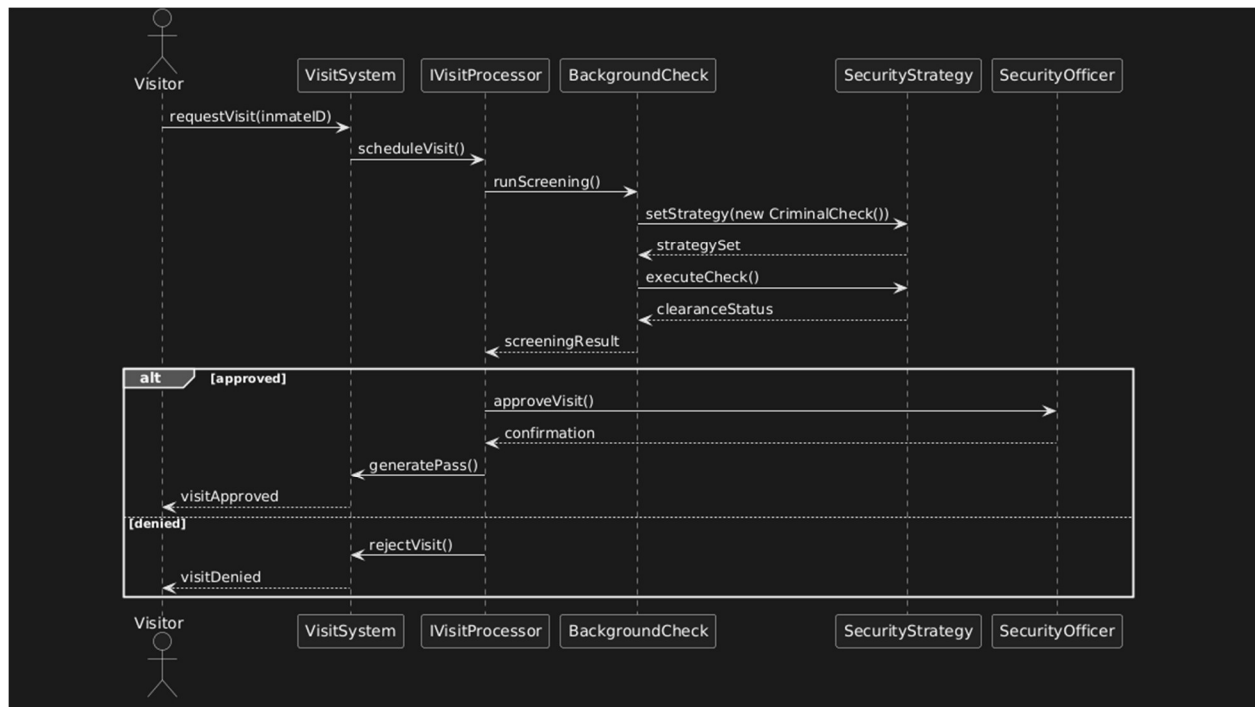
2. Report Generation

Actors: Warden, ReportGenerator, PDFExporter



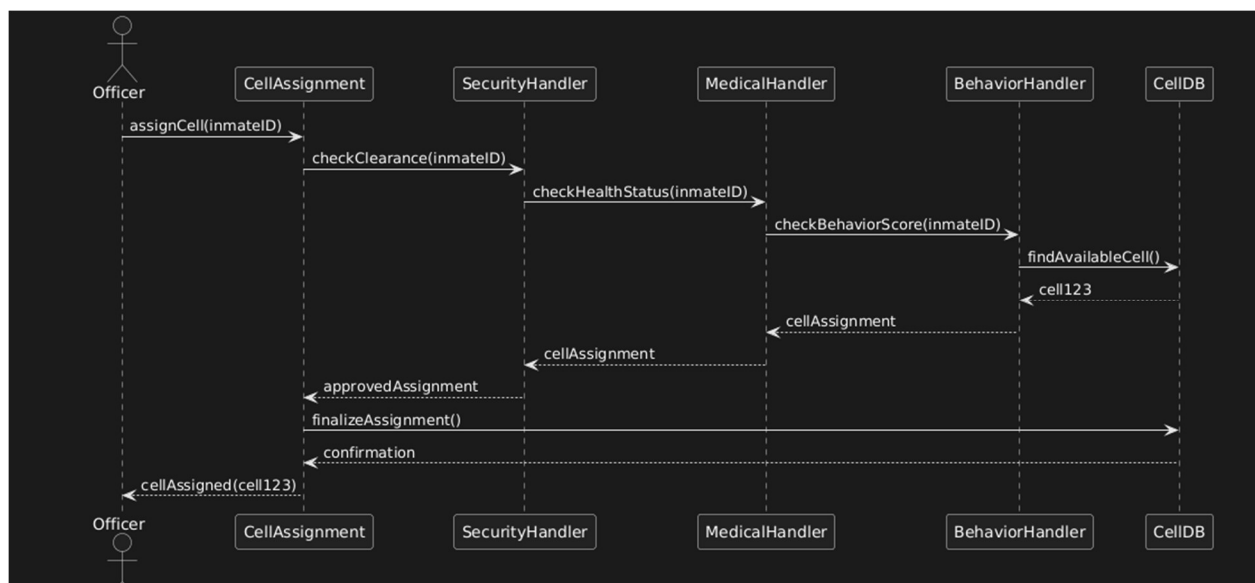
1. Visitor Approval Process

Actors: Visitor, VisitSchedule, SecurityOfficer



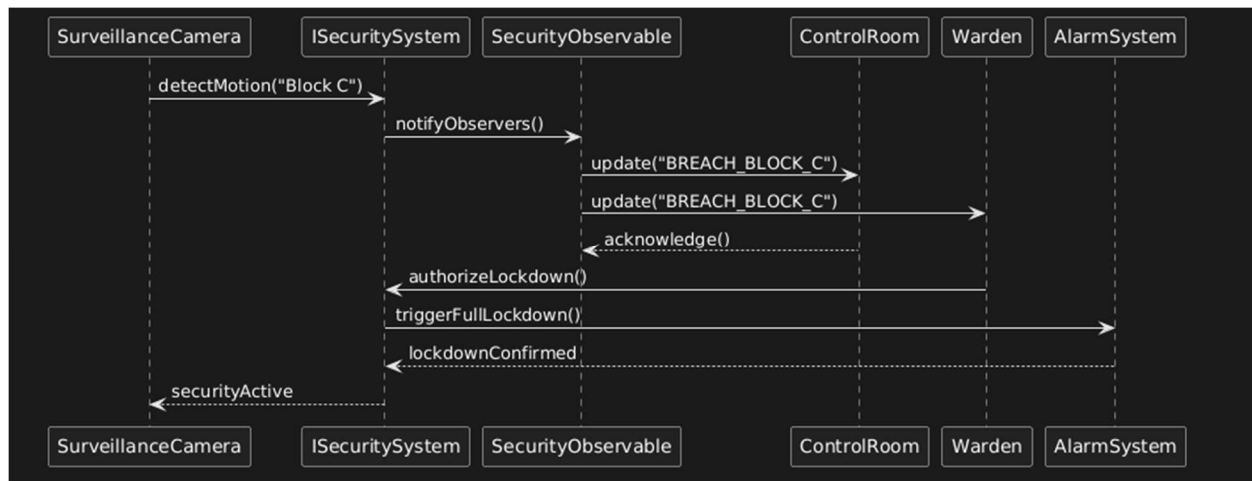
4. Cell Assignment

Actors: CellAssignment, SecurityHandler, MedicalHandler



5. Security Breach Handling

Actors: SurveillanceCamera, AlarmSystem, Warden



Summary

This assignment comprehensively designs a Prison Management System through structural and dynamic modeling. The structural model utilizes class diagrams and four key design patterns—Observer (for real-time alerts), Decorator (for extendable health services), Strategy (for flexible monitoring), and Factory Method (for report generation)—all strictly adhering to defined interfaces like SecurityOperation and MedicalAction. The dynamic model captures critical workflows via sequence diagrams, such as visitor approval (Strategy), inmate registration (Factory), and security breaches (Observer), ensuring alignment with the structural design. The system is documented on GitHub with organized PlantUML files and clear commit messages, demonstrating a cohesive, scalable solution grounded in OOP principles. The implementation reflects meticulous attention to interface contracts, pattern justification, and end-to-end use case coverage.